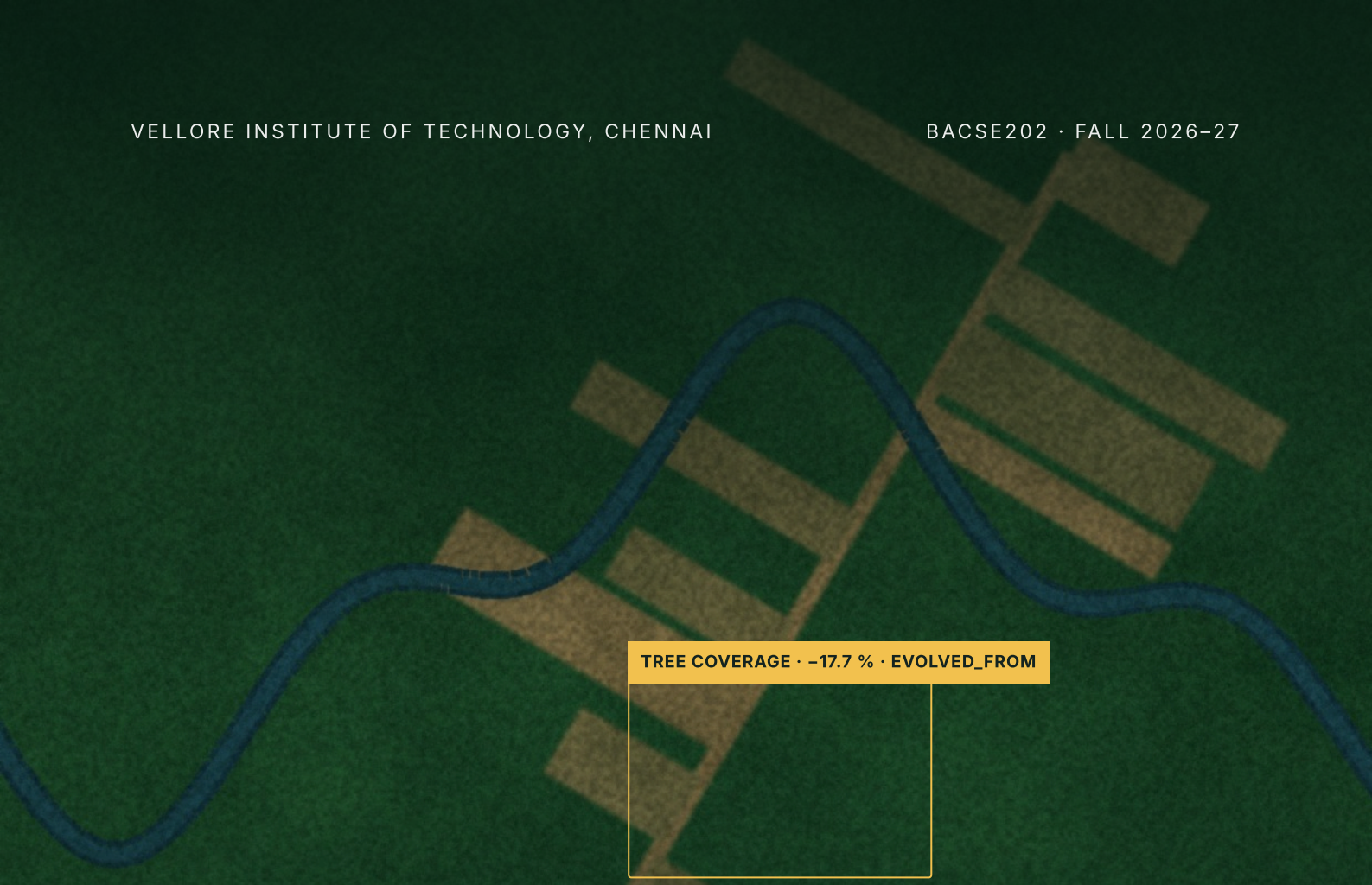
VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

BACSE202 · FALL 2026–27



TECHNICAL PROJECT REPORT · DATABASE MANAGEMENT SYSTEMS

Satellite Imagery Knowledge Graph

GraphRAG for Earth Observation Data

Joshwa D Y
25BCE5176

Nallazhahu M
25BCE5203

Akshay V
25BCE5116

Faculty: Dr. S. Geetha · Slot D1 + TD1 · Class CH202670101707

November 2026

Abstract

Earth-observation (EO) satellites produce images whose metadata (capture time, location, cloud cover, sensor) and AI detections (forest patches, buildings, vessels) are usually stored separately. Connected questions such as "find rapid deforestation near river basins in Q3, and how trustworthy is the evidence?" therefore cannot be answered directly. This project designs and implements a database-backed system that links this data in a 15-table Oracle 21c schema normalised to BCNF. It adds three ideas: temporal lineage that tracks the same forest patch across images; tunable hybrid retrieval that ranks evidence by meaning, distance to a river and date; and confidence propagation that turns detection confidence and cloud cover into a trust score for alerts and answers. PL/SQL packages and triggers enforce the business rules. A Python Flask web application provides nine end-to-end features with full input validation, and a Neo4j extension supports native graph, spatial and vector queries. On synthetic data, Oracle and Neo4j return identical results for the flagship query (10 events) and the trust-aware alerts (4 of 10 events), and all automated tests pass.

15 tables, normalised to BCNF

9 end-to-end application features

4 / 10 deforestation events trusted enough to alert

91 automated tests passed

Keywords: relational database design, BCNF normalization, Oracle PL/SQL, graph database, Neo4j, GraphRAG, Earth observation, deforestation monitoring, vector similarity search, data lineage.

Contents

01 Introduction, problem statement, objectives and scope

02 Existing systems

03 Requirements and proposed system architecture

04 ER diagram and relational schema

05 Keys, constraints, functional dependencies and normalization

06 Implementation: DBMS, application technologies and connectivity

07 Important SQL operations and implemented features

08 Testing, results, screenshots and discussion

09 Conclusion, limitations and future enhancements

— References

01 Introduction

Satellites such as Sentinel-2 and Landsat 8/9 photograph every part of the Earth every few days [4]. AI models then detect objects in these images, and spectral indices such as NDVI measure how green the vegetation is [5]. For forest monitoring, the value lies in *connections*: which patch of forest, near which river, shrank by how much between which two dates, and how reliable were the images that showed it. In practice this information is spread across image catalogues, detection outputs and spreadsheets.

1.1 Problem statement

EO metadata is fragmented. Image timestamps, footprints, cloud cover, sensor details and AI-detected objects are stored separately, with no link between observations of the same place over time. Analysts cannot easily answer: *"Which images show rapid deforestation near river basins in Q3, and how trustworthy is that evidence?"* Answering it needs **what** was detected, **where**, **when**, **how it changed**, and **how reliable** the evidence is.

1.2 Objectives

1. Design a normalised (BCNF) database linking satellites, sensors, images, locations, river basins and detected objects.
2. Track the same real-world object across time (temporal lineage) so that loss and rate of loss can be measured.
3. Store natural-language questions with ranked evidence and an answer-confidence score (a GraphRAG-style log).
4. Raise alerts only for trustworthy changes, and audit every edit to detections.
5. Build a working application with at least five end-to-end features, with validation and error handling.
6. Extend the same model to a graph database (Neo4j) and compare the two approaches.

1.3 Scope

In scope: optical image metadata, detected objects with confidence and area, NDVI/NDWI indices, change tracking, alerts, the question/evidence log, a web application and a graph extension. **Out of scope:** storing raw image pixels (only file paths are stored), real-time satellite downlink, and training detection models. All data is synthetic and generated deterministically by `scripts/generate_data.py` (random seed 2026).

Intended users: EO analysts (search images, study locations, ask questions, review alerts), data/ML engineers (load images and detections, register models) and administrators (manage satellites, users and the audit log).

02 Existing Systems

Several public platforms already provide satellite imagery and forest-change data. They are excellent at viewing imagery and at producing global change layers, but none of them stores an analyst's question together with ranked, trust-scored evidence and the lineage of individual detected objects.

System	URL	What it offers	Gap addressed by this project
Global Nature Watch (formerly Global Forest Watch, World Resources Institute)	https://www.globalnaturewatch.org	Interactive world map of forest change, land cover, land use, climate and biodiversity layers, with near-real-time deforestation alerts built on the Hansen global forest-change data [3], [12], [13]	Alerts are pixel layers; no per-object lineage, no stored questions with ranked evidence or answer confidence
Copernicus Browser (Copernicus Data Space Ecosystem)	https://browser.dataspace.copernicus.eu	Search, visualise, compare and download Sentinel imagery, with band combinations and index visualisations such as NDVI [14]	An image viewer; detections, change history and trust are left to the user
Google Earth Engine	https://earthengine.google.com	Planetary-scale catalogue and cloud computation for geospatial analysis via code [7]	Powerful but programmatic; no relational model of detections, alerts and audit
NASA Worldview	https://worldview.earthdata.nasa.gov	Browsing of near-real-time global imagery layers by date	Visual browsing only; no object-level database

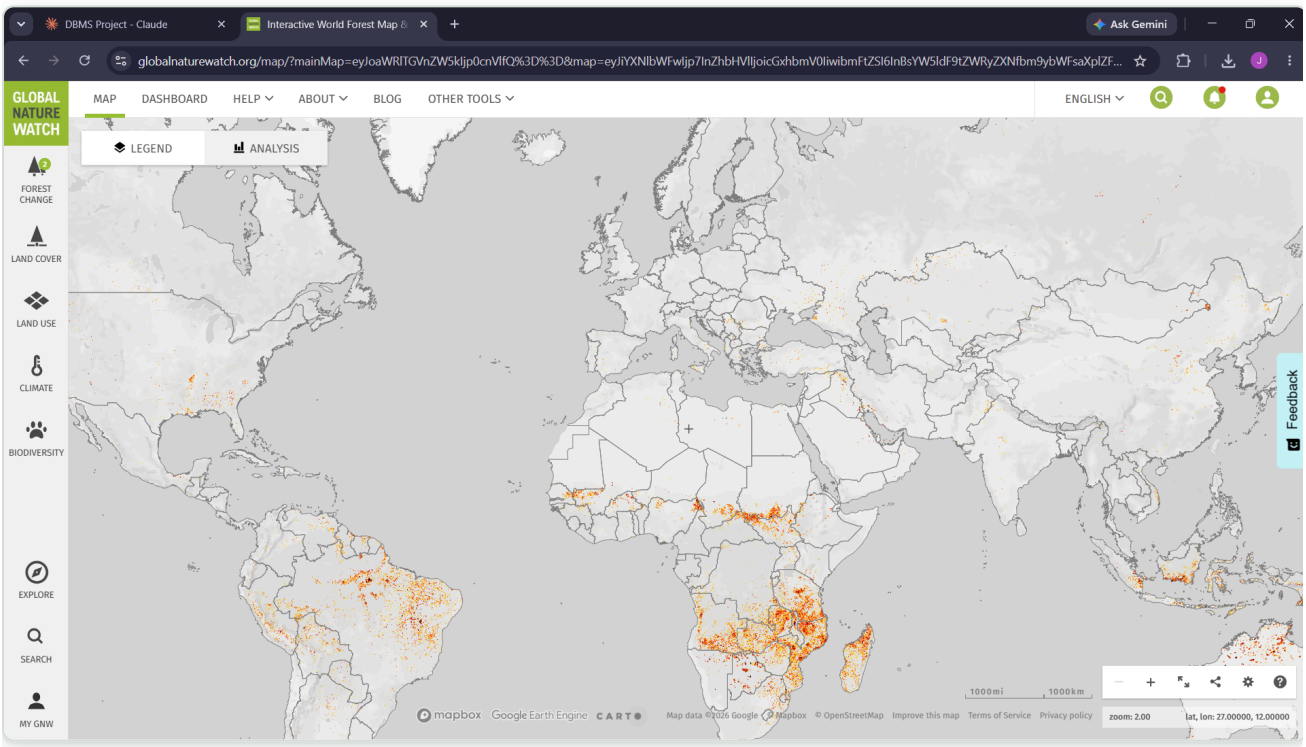


Figure 1. Global Nature Watch: world forest map with tree-cover-loss and alert layers (globalnaturewatch.org/map, captured 1 Nov 2026).

Positioning. This project does not replace these platforms; it is the database layer that could sit beneath them. It stores detections as rows linked to images, places and basins; it links the same patch across dates; and it records each question with its evidence, weights and confidence. GraphRAG [1], which combines a knowledge graph with retrieval-augmented generation [2], motivates the question/evidence design.

03 Requirements and Proposed System Architecture

3.1 Functional requirements

ID	Requirement
FR1	Register satellites and the sensors they carry (create, read, update, delete).
FR2	Record each image with its sensor, location, capture time, cloud cover and file path; search and filter images.
FR3	Record objects detected in each image: type, model, confidence, bounding box, area, attributes.
FR4	Store NDVI and NDWI spectral indices per image.
FR5	Link each location to its river basin (optional for coastal and urban places).
FR6	Link two observations of the same object at different times (EVOLVED_FROM) and store the change, days and rate.
FR7	Raise one alert per significant, trustworthy change; assign and resolve alerts.
FR8	Accept a question, rank evidence by meaning, distance and time, store it with weights, and compute an answer confidence.
FR9	Audit inserts, updates and deletes of detections automatically.

Non-functional requirements: data integrity enforced by the DBMS (not only the application); SQL-injection safety through bind variables; readable error messages for every constraint violation; reproducible setup (scripts run in order, deterministic data); no secrets in the repository (.env excluded).

3.2 Architecture

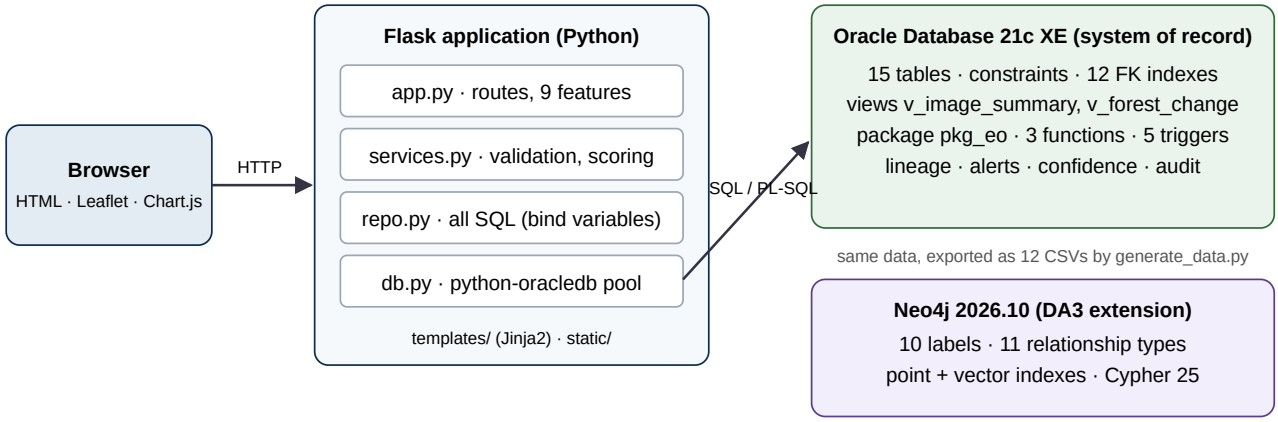


Figure 2. System architecture: browser, layered Flask application, Oracle (system of record) and the Neo4j graph extension.

The application follows a layered design: **app.py** handles HTTP routes; **services.py** holds validation, scoring and error translation (no database access, so it is unit-testable); **repo.py** holds every SQL statement; **db.py** manages the connection pool. Business rules that must never be bypassed (alerts, lineage, confidence, audit) live inside Oracle as PL/SQL. Neo4j holds the same data as a graph for graph, spatial and vector queries.

04 ER Diagram and Relational Schema

The design has **15 entities**: 11 strong entities, 2 weak entities (SENSOR, owned by SATELLITE with partial key sensor_name; SPECTRAL_INDEX, owned by IMAGE_FRAME with partial key index_name) and 2 associative entities (OBJECT_CHANGE for the recursive EVOLVED_FROM relationship, and QUERY_EVIDENCE for the M:N relationship between questions and detected objects). It grew from a first 8-entity design: SENSOR was added because resolution belongs to the instrument, and basin_id was removed from DETECTED_OBJECT because it follows from image → location → basin.

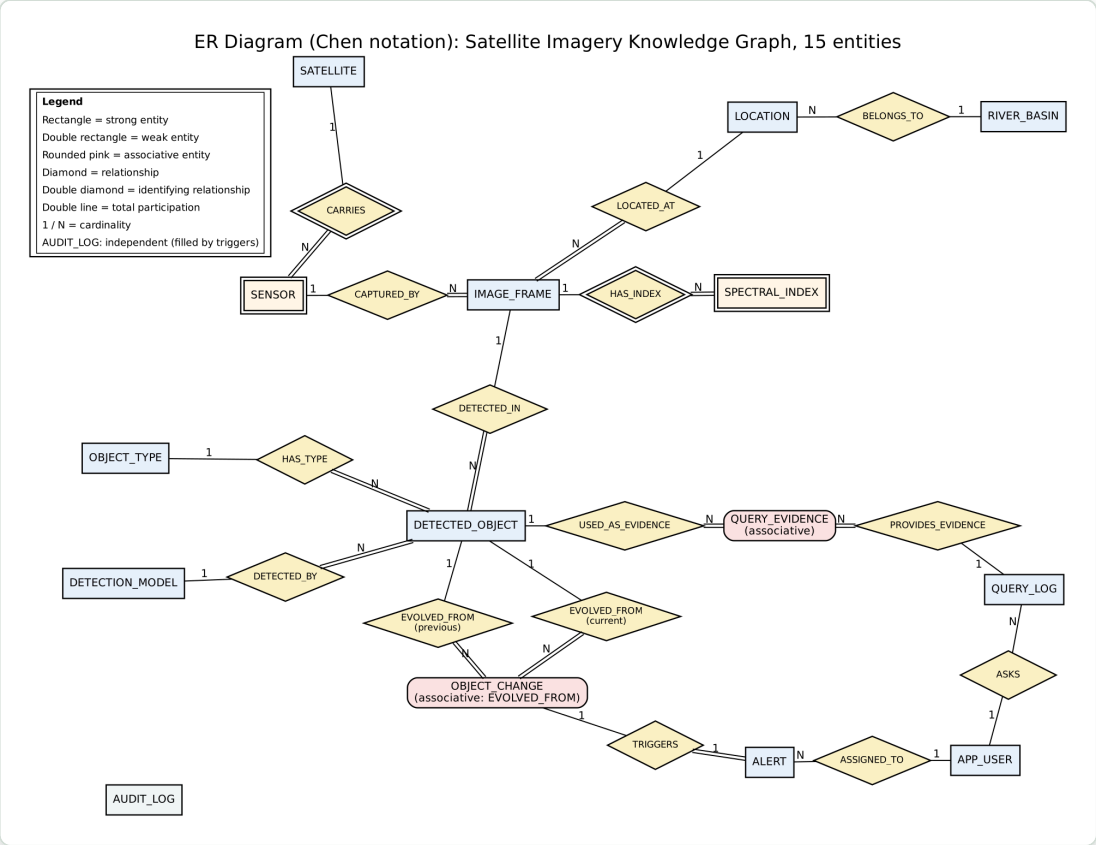


Figure 3. ER diagram (Chen notation). Double rectangles are weak entities, double diamonds identifying relationships, double lines total participation.

Relationship	Entities	Cardinality	Participation
CARRIES (identifying)	SATELLITE – SENSOR	1 : N	SENSOR total
CAPTURED_BY / LOCATED_AT	SENSOR, LOCATION – IMAGE_FRAME	1 : N	IMAGE total
BELONGS_TO	RIVER_BASIN – LOCATION	1 : N	both partial
HAS_INDEX (identifying)	IMAGE_FRAME – SPECTRAL_INDEX	1 : N	INDEX total
DETECTED_IN / HAS_TYPE / DETECTED_BY	IMAGE_FRAME, OBJECT_TYPE, DETECTION_MODEL – DETECTED_OBJECT	1 : N	OBJECT total
EVOLVED_FROM (recursive)	DETECTED_OBJECT – DETECTED_OBJECT via OBJECT_CHANGE	1 : N per role	first sighting partial
PROVIDES_EVIDENCE	QUERY_LOG – DETECTED_OBJECT via QUERY_EVIDENCE	M : N	both partial

ASKS / ASSIGNED_TO	APP_USER – QUERY_LOG, ALERT	1 : N	both partial
TRIGGERS	OBJECT_CHANGE – ALERT	1 : 1	ALERT total

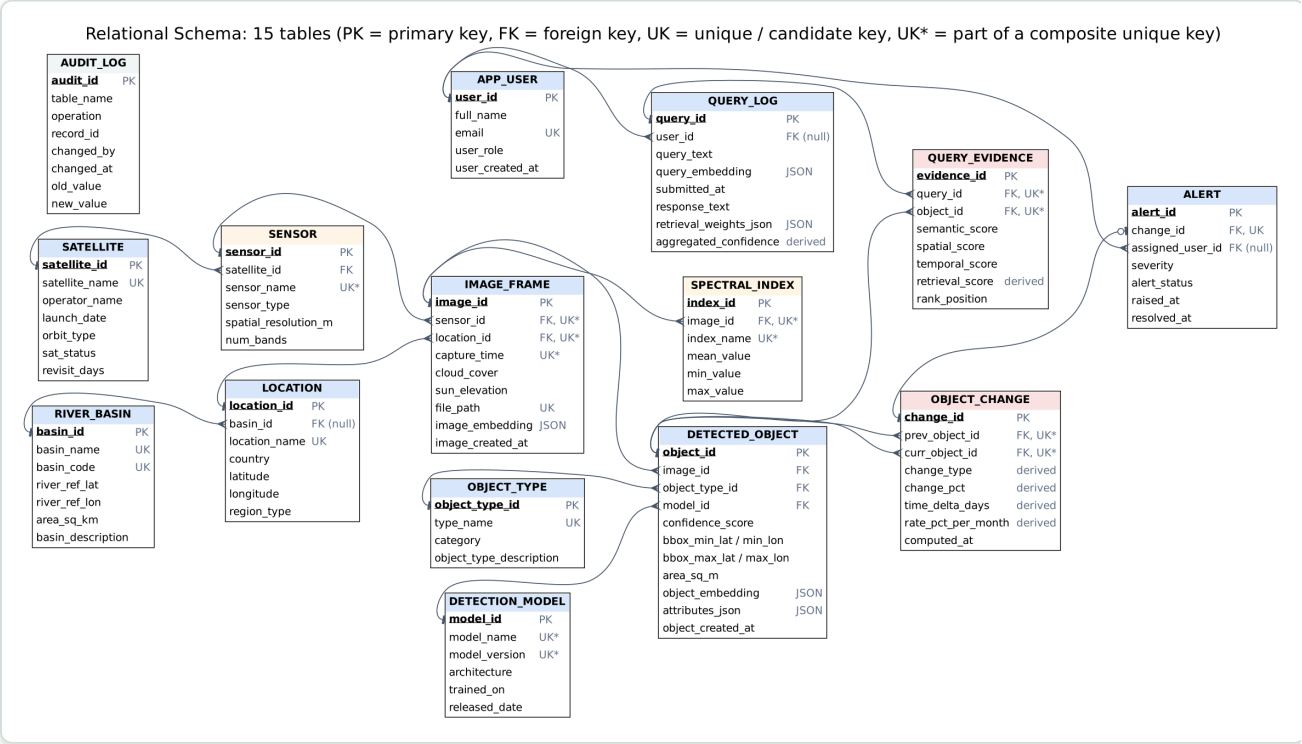


Figure 4. Relational schema: the 15 tables with primary keys and foreign keys.

Mapping rules used: each strong entity became a table; each weak entity became a table with the owner's key as a foreign key and UNIQUE(owner, partial key), plus a surrogate key; each 1:N relationship became a foreign key on the N side (NOT NULL when participation is total); the 1:1 TRIGGERS relationship became a UNIQUE foreign key change_id in ALERT; the M:N relationship and the recursive relationship with attributes became their own tables; composite attributes were flattened (latitude/longitude, four bounding-box columns); multi-valued embeddings were stored as one JSON value with an IS JSON check.

05 Keys, Constraints, Functional Dependencies and Normalization

5.1 Keys

Table	Primary key	Candidate / unique keys	Foreign keys
SATELLITE	satellite_id	satellite_name	–
SENSOR	sensor_id	(satellite_id, sensor_name)	satellite_id
RIVER_BASIN	basin_id	basin_code; basin_name	–
LOCATION	location_id	location_name	basin_id (nullable)
IMAGE_FRAME	image_id	file_path; (sensor_id, location_id, capture_time)	sensor_id, location_id
OBJECT_TYPE / DETECTION_MODEL	object_type_id / model_id	type_name / (model_name, model_version)	–
DETECTED_OBJECT	object_id	–	image_id, object_type_id, model_id
OBJECT_CHANGE	change_id	(prev_object_id, curr_object_id)	prev_object_id, curr_object_id
SPECTRAL_INDEX	index_id	(image_id, index_name)	image_id
APP_USER	user_id	email	–
QUERY_LOG	query_id	–	user_id (nullable)
QUERY_EVIDENCE	evidence_id	(query_id, object_id)	query_id, object_id
ALERT	alert_id	change_id	change_id, assigned_user_id (nullable)
AUDIT_LOG	audit_id	–	– (written by triggers)

5.2 Integrity constraints

- **Domain (CHECK):** cloud_cover 0–100; sun_elevation 0–90; confidence_score 0–1; area_sq_m > 0; latitude −90…90, longitude −180…180; spectral min ≤ mean ≤ max within −1…1; scores 0–1; time_delta_days > 0.
- **Enumerations (CHECK … IN):** orbit_type, sat_status, sensor_type, region_type, index_name, change_type, severity, alert_status, user_role, audit operation.
- **Structural:** bounding-box min < max; prev_object_id ≠ curr_object_id; resolved_at only when status is RESOLVED; JSON columns IS JSON.
- **Referential:** ON DELETE CASCADE for dependent rows (sensor, object, index, change, evidence, alert); SET NULL for optional links; RESTRICT where history would be lost (a sensor with images).
- **Trigger-enforced:** capture_time cannot be in the future; an EVOLVED_FROM pair must be at the same location with the older object first.

5.3 Functional dependencies

Starting relation (UNF): one flat record per image, `EO_RECORD(file_path, capture_time, cloud_cover, sun_elevation, satellite…, sensor…, location…, basin…, {object_no, type, model, confidence, bbox, area, attributes}*, {index_name, mean, min, max}*)`, with two repeating groups and update, insert and delete anomalies (for example, a basin's area repeated in every image row).

#	Functional dependency
FD1	satellite_name → operator_name, launch_date, orbit_type, sat_status, revisit_days
FD2	(satellite_name, sensor_name) → sensor_type, spatial_resolution_m, num_bands
FD3	file_path → capture_time, cloud_cover, sun_elevation, satellite_name, sensor_name, location_name
FD4	(satellite_name, sensor_name, location_name, capture_time) → file_path
FD5	location_name → country, latitude, longitude, region_type, basin_code
FD6	basin_code → basin_name, river_ref_lat, river_ref_lon, area_sq_km; basin_name → basin_code
FD7	type_name → category, object_type_description
FD8	(model_name, model_version) → architecture, trained_on, released_date
FD9	(file_path, object_no) → type_name, model, confidence, bbox, area_sq_m, attributes
FD10	(file_path, index_name) → mean_value, min_value, max_value

5.4 Normalization steps

Form	Rule	Decomposition
1NF	Atomic values, no repeating groups	bbox split into 4 columns; one row per object (key file_path, object_no) and per index (key file_path, index_name)
2NF	No partial dependency on part of a composite key	Image, satellite, sensor, location and basin columns depend only on file_path → moved to an IMAGE relation
3NF	No transitive dependency	file_path → satellite → operator, → sensor → resolution, → location → basin; type → category; model → architecture: split into SATELLITE, SENSOR, LOCATION, RIVER_BASIN, OBJECT_TYPE, DETECTION_MODEL
BCNF	Every determinant is a superkey	All 9 pipeline relations pass; IMAGE_FRAME has two overlapping candidate keys (file_path and FD4's composite), both superkeys

The decomposition is **lossless** (each split shares a key) and **dependency-preserving** (each FD lies inside one relation). Natural keys were then replaced by numeric surrogate keys and kept as UNIQUE constraints. The six application tables (APP_USER, QUERY_LOG, QUERY_EVIDENCE, OBJECT_CHANGE, ALERT, AUDIT_LOG) are in BCNF, giving 15 relations.

Justified denormalization. OBJECT_CHANGE stores change_pct, time_delta_days, rate_pct_per_month and change_type although they can be derived from two objects' areas and capture times, and ALERT.severity, QUERY_EVIDENCE.retrieval_score and QUERY_LOG.aggregated_confidence are also stored. Each is a historical snapshot (an alert must show the value that triggered it; re-tuning weights must not rewrite old answers) and each is written only by one PL/SQL procedure, so the values cannot drift. Embeddings are stored as JSON because they are always read and written as whole vectors.

06 Implementation

6.1 DBMS

Oracle Database 21c Express Edition (pluggable database XEPDB1, schema JOSHWA), managed with Oracle SQL Developer 26.2. Six scripts are run in order:

Script	Contents	Observed result
schema.sql	Drop-and-recreate block; 15 tables with all constraints; identity columns (START WITH 10000); 12 FK indexes; views v_image_summary and v_forest_change	15 tables, 12 indexes, 2 views created
sample_data.sql	~490 INSERTs from the generator	247 detected objects, 60 images, 120 indices
plsql.sql	fn_haversine_km, fn_classify_change, fn_forest_loss_pct; package pkg_eo (compute_changes, raise_alerts, fn_query_confidence, finalize_query, add_detection, basin_report); 4 triggers	all compiled, no errors
run_procedures.sql	Runs the package and demonstrates the triggers	36 lineage links; 4 alerts; confidences 61.2 / 66.6 / 61.8 / 60.1 %
queries.sql	Q1–Q20 (Section 7)	results match independently predicted values
extras.sql	ALTER, GRANT/REVOKE, explicit cursors, FOR UPDATE, user-defined exception, IN/OUT procedure, statement-level trigger, cascading DELETE with rollback, data dictionary	E1–E9 as expected

The key PL/SQL procedure `compute_changes` pairs each tree patch with the previous one at the same location using the LAG window function, computes change % = (previous − current) / previous × 100 and rate = change % ÷ days × 30, classifies it, and stores it with MERGE so re-running never duplicates. `raise_alerts` creates an alert only when loss ≥ 10 %, both detections have confidence ≥ 0.7 and both images have cloud ≤ 40 %. `fn_query_confidence` computes:

```
confidence = Σ (retrieval_score × detection_confidence × (1 − cloud/100)) ÷ Σ
                                retrieval_score
```

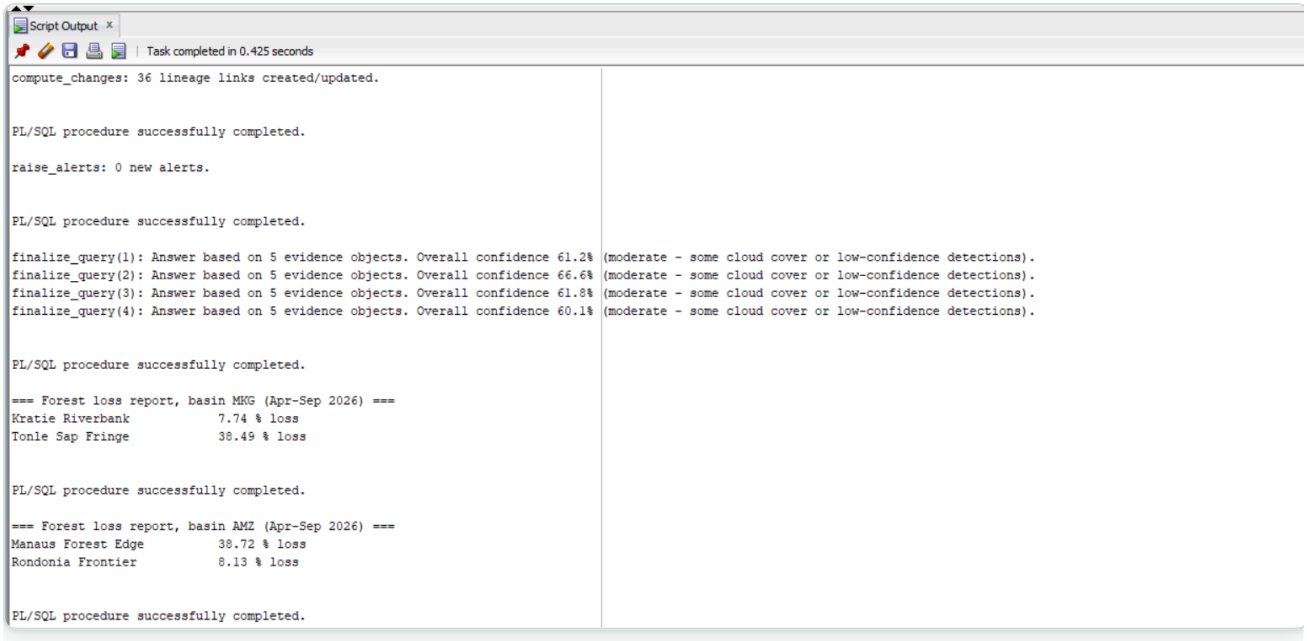


Figure 5. run_procedures.sql output in SQL Developer: 36 lineage links, 4 alerts, answer confidences and basin reports.

6.2 Application technologies

Python 3, Flask 3 (web framework) [15], Jinja2 templates, python-oracledb 2 in thin mode (no Oracle Client install) [16], python-dotenv for configuration, Leaflet 1.9.4 for maps and Chart.js 4.4.1 for charts, and pytest for testing.

6.3 Database connectivity

- **Connection pool:** db.py creates a python-oracledb pool (1–4 connections) from the .env settings (user, password, DSN localhost:1521/XEPDB1); each request borrows a connection and returns it.
- **Bind variables everywhere:** every statement in repo.py uses named binds (`:min_loss`, `:d_from`…); user input is never concatenated into SQL, preventing SQL injection.
- **Calling PL/SQL:** `cursor.callproc("pkg_eo.add_detection", [...])` with an OUT variable for the new id; DBMS_OUTPUT is read back and shown on the Alerts page.
- **Transactions:** a question and its five evidence rows are inserted in one transaction (INSERT … RETURNING query_id, executemany, COMMIT) before finalize_query computes the confidence.
- **Error translation:** Oracle errors are mapped by code and constraint name to readable messages (ORA-00001, 02290, 02291, 02292, 01400, 20xxx, connection errors).

6.4 Graph extension (Neo4j)

The same dataset is loaded into Neo4j 2026.10 (Cypher 25) from 12 CSV files: 354 nodes with 10 labels and 11 relationship types. OBJECT_CHANGE becomes an EVOLVED_FROM relationship with properties; QUERY_EVIDENCE becomes USED_AS_EVIDENCE; coordinates become spatial points with point indexes; embeddings become lists with an 8-dimensional cosine vector index [17]. Fourteen technology-specific queries (G1–G14) and nine CRUD steps (C1–C9) were run.

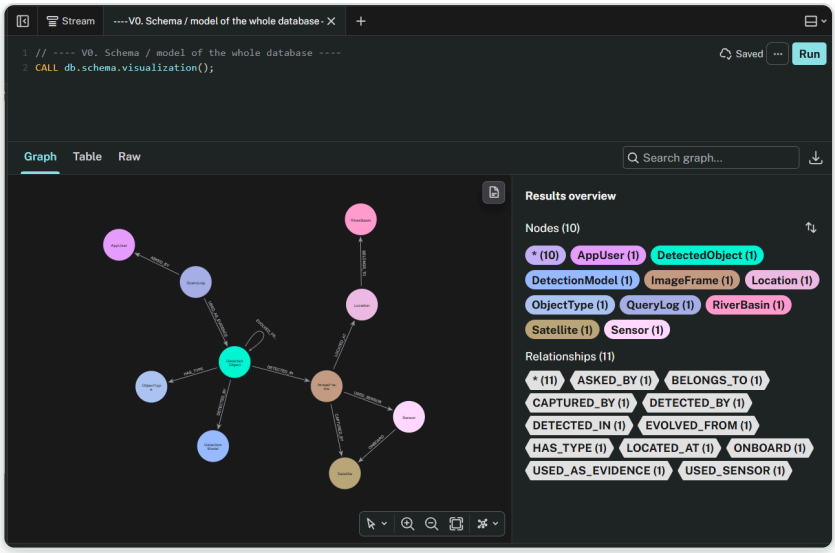


Figure 6. Neo4j graph model (db.schema.visualization()): 10 labels and 11 relationship types.

07 Important SQL Operations and Implemented Features

7.1 SQL queries (queries.sql)

Q	Question answered	SQL feature	Result
Q1	Images with satellite, sensor and location	view (5-table join), FETCH FIRST	10 rows
Q2	Locations outside any basin	LEFT JOIN anti-join	Chennai, Rotterdam, Dubai
Q4	Images and average cloud per satellite	JOIN, GROUP BY, AVG	4 × 15 images
Q5	Detections per type, more than 5	GROUP BY … HAVING	7 types
Q7 / Q8	Cloudier than average; latest tree area	non-correlated / correlated subquery	8 rows / 9 rows
Q9	Deforestation without an alert	MINUS	Manaus, Bhadrachalam
Q10	**Rapid deforestation near river basins in Q3**	view + PL/SQL function in WHERE	**10 events, max 16.15 %/month**
Q11	Same events ranked by evidence trust	LEAST, GREATEST	0.584 … 0.306
Q12	Cumulative loss and rank	SUM() OVER, RANK() OVER	36 rows
Q13	Full lineage chain of each patch	CONNECT BY, SYS_CONNECT_BY_PATH	e.g. 33→35→37→39→41
Q14	NDVI vs tree area	CORR	0.442
Q15	Vessels by class	JSON_VALUE	cargo 11, tanker 9…
Q17 / Q18	Evidence per question; alert dashboard	6-table join; ORDER BY CASE	12 rows / 4 alerts
Q19	Transaction demo	SAVEPOINT, UPDATE, ROLLBACK TO	5 rows updated, rolled back
Q20	Integrity violations	invalid INSERT / DELETE	ORA-02290, 02291, 02292

```
SELECT fc.location_name, fc.basin_name, fc.curr_time, fc.change_pct, fc.rate_pct_per_month,
       ROUND(fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon)) AS
km_to_river
FROM v_forest_change fc JOIN location l ON l.location_id = fc.location_id
JOIN river_basin b ON b.basin_id = l.basin_id
WHERE fc.change_type = 'DEFORESTATION'
  AND fc.curr_time >= TIMESTAMP '2026-07-01 00:00:00' AND fc.curr_time < TIMESTAMP '2026-10-01
00:00:00'
  AND fn_haversine_km(l.latitude, l.longitude, b.river_ref_lat, b.river_ref_lon) <= 300
ORDER BY fc.rate_pct_per_month DESC;
```

Script Output × Query Result ×

SQL | All Rows Fetched: 10 in 0.064 seconds

	LOCATION_NAME	BASIN_NAME	CURR_TIME	CHANGE_PCT	RATE_PCT_PER_MONTH	KM_TO_RIVER
1	Bhadrachalam Forest	Godavari Basin	29-SEP-26 05.30.00.000000000 AM	17.76	16.15	123
2	Manaus Forest Edge	Amazon Basin	29-SEP-26 05.30.00.000000000 AM	17.77	16.15	65
3	Kisangani Rainforest	Congo Basin	29-SEP-26 05.30.00.000000000 AM	17.7	16.09	38
4	Tonle Sap Fringe	Mekong Basin	29-SEP-26 05.30.00.000000000 AM	17.41	15.83	174
5	Kisangani Rainforest	Congo Basin	27-AUG-26 05.30.00.000000000 AM	14.48	12.07	38
6	Manaus Forest Edge	Amazon Basin	27-AUG-26 05.30.00.000000000 AM	14.3	11.92	65
7	Tonle Sap Fringe	Mekong Basin	27-AUG-26 05.30.00.000000000 AM	13.91	11.59	174
8	Bhadrachalam Forest	Godavari Basin	27-AUG-26 05.30.00.000000000 AM	13.5	11.25	123
9	Kisangani Rainforest	Congo Basin	22-JUL-26 05.30.00.000000000 AM	10.78	9.51	38
10	Tonle Sap Fringe	Mekong Basin	22-JUL-26 05.30.00.000000000 AM	10.66	9.41	174

Figure 7. Q10, the flagship query: 10 deforestation events within 300 km of a river in Q3, fastest first.

7.2 Implemented application features

#	Feature	What it does	Database operations
1	Dashboard	KPIs, forest loss by location, open alerts, recent questions	scalar subqueries, fn_forest_loss_pct
2	Image catalogue	Filter by satellite, location, cloud, dates; detail with objects and indices	SELECT on view with optional bind filters
3	Location profile	Map; tree-area and NDVI chart; lineage chain; change history	SELECT on OBJECT_CHANGE, SPECTRAL_INDEX
4	Deforestation finder	The flagship question as a form with map and trust score	Q10 with bind variables
5	Change analysis and alerts	Run analysis with thresholds; assign and resolve alerts	CALL compute_changes, raise_alerts; UPDATE + trigger
6	Ask (GraphRAG)	Question → ranked evidence → answer confidence	INSERT log + evidence (one transaction); CALL finalize_query
7	Add detection	Validated insert; duplicate tree patch blocked	CALL add_detection (SAVEPOINT, OUT id); audit trigger
8	Satellites CRUD	Create, edit, delete with friendly errors	INSERT, UPDATE, DELETE; UNIQUE / FK errors
9	Audit log	Rows written only by triggers	SELECT on AUDIT_LOG

Validation and error handling happen at two levels. The application rejects invalid input before touching the database (e.g. confidence 1.5, an empty satellite name, a future launch date, all weights zero, a "from" date after the "to" date). The database enforces the same rules through constraints and triggers. Missing records return a 404 page, duplicates (a second satellite with the same name, a second tree patch on an image) are refused with clear messages, and blocked deletes (a satellite that still has images) explain why.

7.3 Hybrid retrieval (Ask)

The question's concept is turned into a vector (the centroid of labelled example embeddings). Every detected object is then scored and the top k (3–10) are kept:

$$\text{retrieval} = \alpha\cdot\text{semantic} + \beta\cdot\text{spatial} + \gamma\cdot\text{temporal},\quad \text{semantic} = \text{cosine}(q, e),\quad \text{spatial} = e^{-\text{km}/200},\quad \text{temporal} = 1.0 \text{ in window, else } 0.3$$

For sample question 1 (α 0.5, β 0.3, γ 0.2) the top evidence is a Kisangani tree patch (semantic 0.9995, spatial 0.8289 at 38 km, temporal 1.0, retrieval 0.9484), and Oracle computes an answer confidence of **0.6121**: moderate, because some evidence images were cloudy.

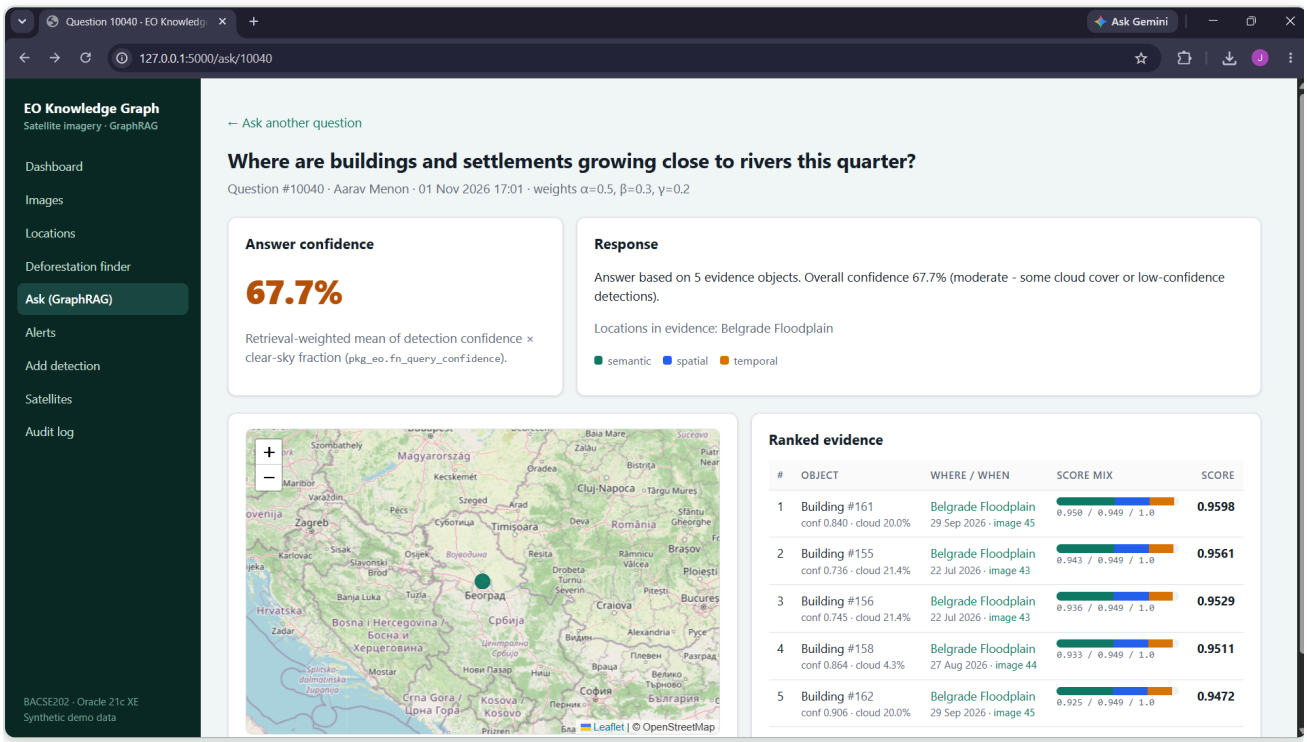


Figure 8. Ask (GraphRAG) result for a new question, "Where are buildings and settlements growing close to rivers this quarter?": five Belgrade Floodplain buildings 10 km from the Danube, each with its semantic / spatial / temporal score mix, and an answer confidence of 67.7 % computed in Oracle by pkg_eo.fn_query_confidence.

08 Testing, Results, Screenshots and Discussion

8.1 Testing approach

Level	File	What is checked
Unit	tests/test_services.py	validators, cosine, haversine (Chennai–Delhi ≈ 1,757 km), ranking, the confidence formula, Oracle error translation
UI (mocked database)	tests/test_app_smoke.py	every page renders; every form path: valid, invalid, duplicate, blocked delete, database down
Integration (live Oracle)	tests/test_integration_db.py, tests/test_integration_app.py	data loaded; flagship query = 10 rows; CRUD round trip; ORA-00001/02290/02292; package error 20020; duplicate tree patch; audit and resolve triggers; GraphRAG end to end

Running `python -m pytest` on the project laptop (Windows, Python 3.14, Oracle 21c XE) collected and passed **all 91 tests in 3.32 s**: 32 unit tests, 29 UI tests with a mocked database, 18 live UI tests and 12 live database tests. The full test-case table with expected and actual outcomes is in `tests/TEST_CASES.md`; a selection is shown below.

ID	Input	Expected	Actual
TC01	Run the four setup scripts	36 links, 4 alerts, confidences 0.6121/0.6655/0.618/0.6006	Pass
TC04	/images?max_cloud=150	"Max cloud % must be at most 100", no crash	Pass
TC06	/images/999 (missing record)	"Record not found", HTTP 404	Pass
TC08	Finder with defaults	10 events at Manaus, Kisangani, Tonle Sap, Bhadrachalam	Pass
TC12	Set an alert to RESOLVED	resolved_at filled by trigger	Pass
TC13	Ask with defaults	5 evidence rows saved; confidence in [0, 1]	Pass
TC16	Detection confidence 1.5, area −1	Two validation errors, nothing saved	Pass
TC17	Second Tree Coverage on image 1	Duplicate rejected	Pass
TC21	Create "Sentinel-2A" again	ORA-00001 → "A satellite with this name already exists."	Pass
TC24	Delete Sentinel-2A	ORA-02292 → "Cannot delete: other records still depend on this one"	Pass
TC25	cloud_cover = 140	ORA-02290 → "Cloud cover must be between 0 and 100%."	Pass
TC28	/images?location=x' OR '1'='1	Treated as text: no rows, no error	Pass

8.2 Results

- 36 lineage links; 10 deforestation events, of which only **4** passed the trust rules and became alerts (the other 6 had cloud > 40 % or confidence < 0.7).

- Rapidly deforesting sites lost about 38–39 % of tree area in six months (Kisangani 39.1 %, Manaus 38.7 %, Tonle Sap 38.5 %, Bhadrachalam 38.0 %); slow sites about 8 %; stable sites changed by less than 1 %.
- NDVI correlates positively with detected tree area (r = 0.442), as expected physically.
- **Oracle and Neo4j agree:** Q10 and Cypher G3 return the same 10 rows; raise_alerts and G8 return the same 4 changes; CONNECT BY and the EVOLVED_FROM* path return the same chains.
- All outputs matched values predicted independently by the Python data generator (e.g. confidences to four decimals).

8.3 Screenshots

The screenshots below were taken from the running application connected to Oracle 21c XE.

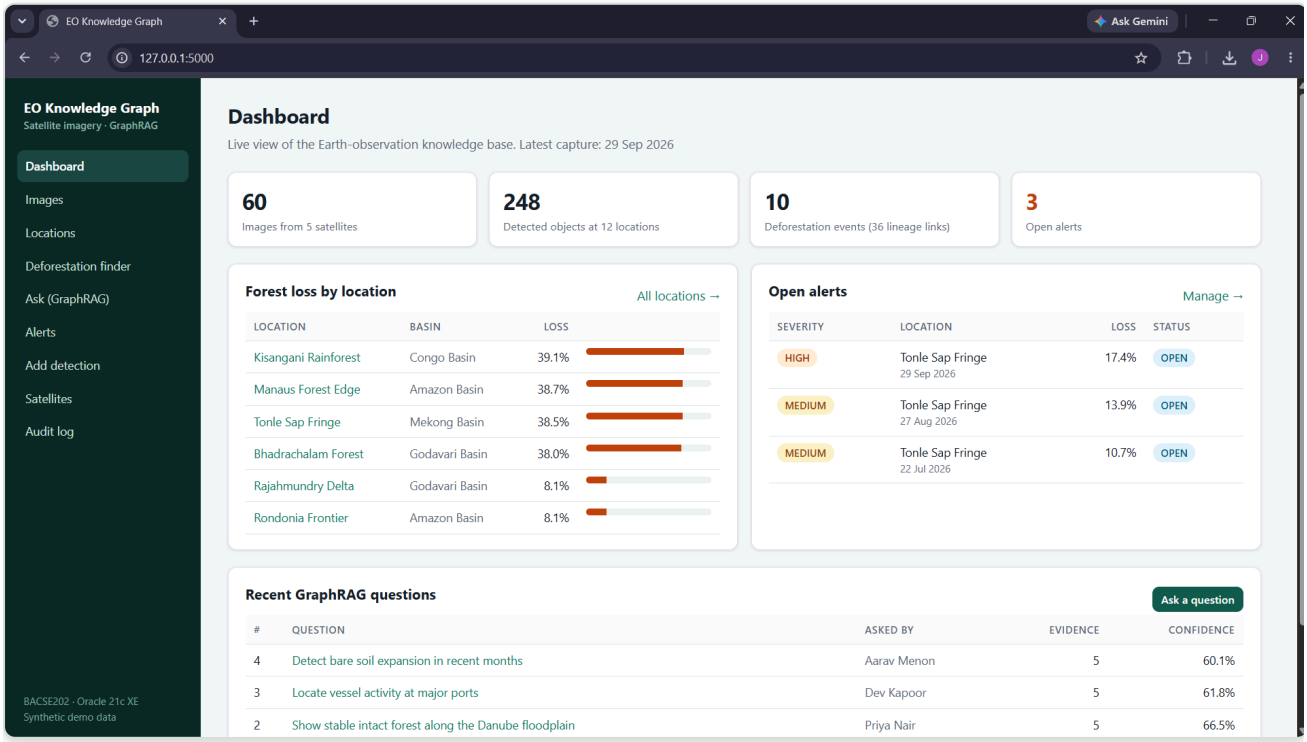


Figure 9. Dashboard: 60 images from 5 satellites, 248 detected objects, 10 deforestation events from 36 lineage links, 3 open alerts, forest loss ranked by location and recent GraphRAG questions.

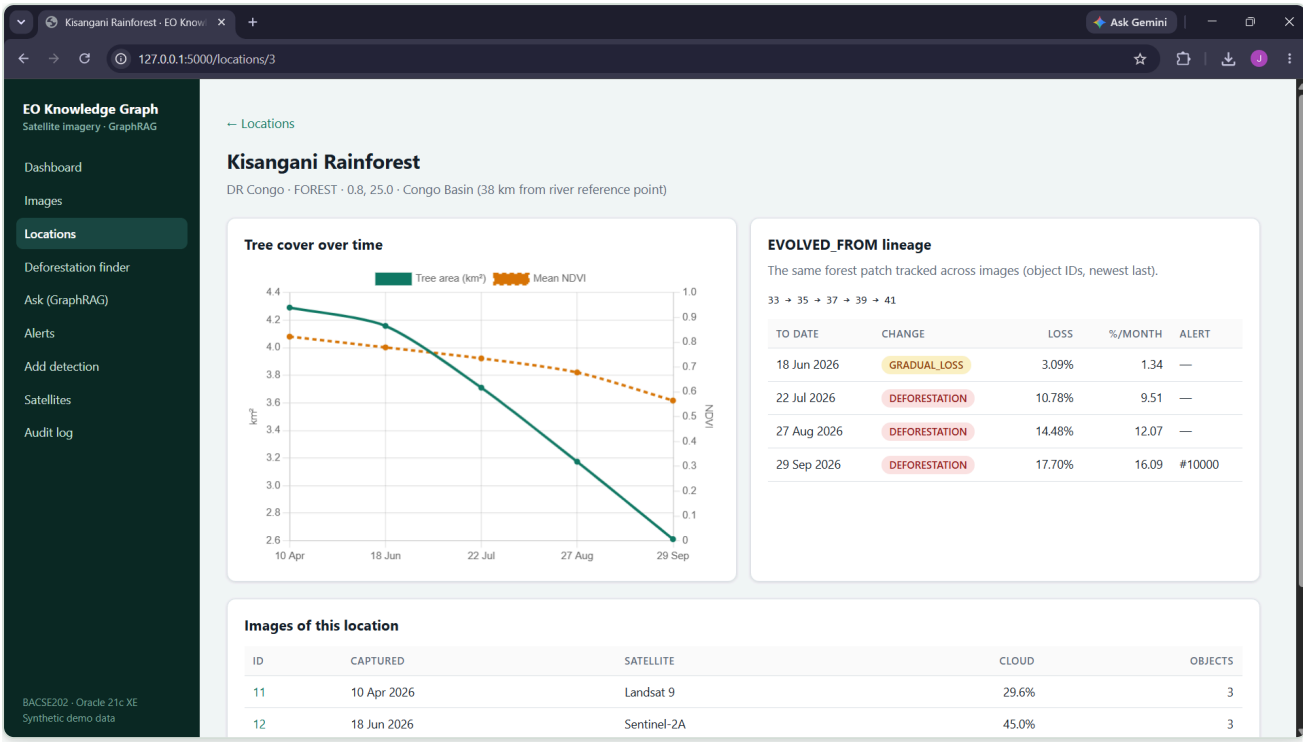


Figure 10. Location profile, Kisangani Rainforest: tree area falls from 4.29 to 2.61 km² while mean NDVI falls with it; the EVOLVED_FROM chain 33 → 35 → 37 → 39 → 41 and the change history, with alert #10000 on the last step.

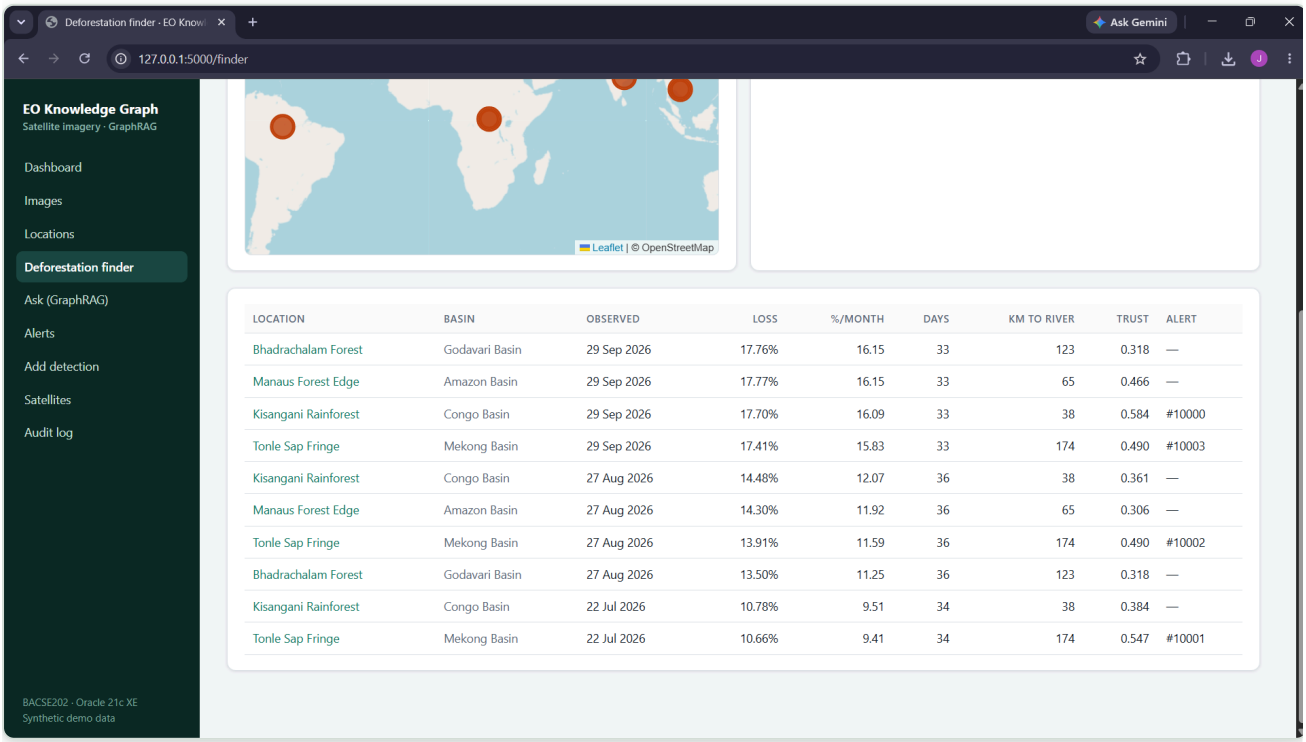


Figure 11. Deforestation finder: the flagship question returns the same 10 events as SQL Q10 and Cypher G3, with rate per month, distance to the river, evidence trust and linked alerts.

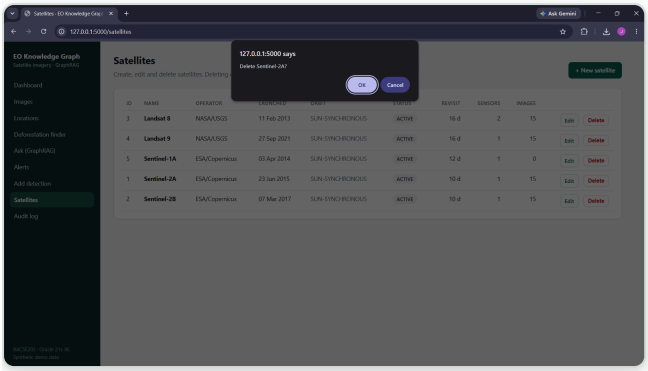


Figure 12. Delete request for Sentinel-2A, which still has 15 images.

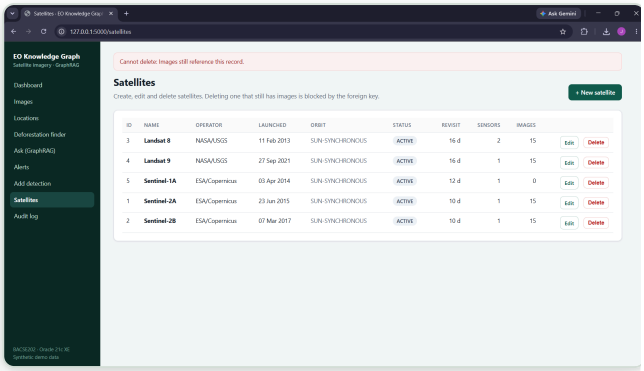


Figure 13. Constraint violation handled: ORA-02292 shown as "Cannot delete: Images still reference this record."

Script Output | Query Result

SQL | All Rows Fetched: 4 in 0.028 seconds

	ALERT_ID	SEVERITY	LOCATION_NAME	BASIN_NAME	CHANGE_PCT	ALERT_STATUS	ASSIGNEE
1	10000	HIGH	Kisangani Rainforest	Congo Basin	17.7	RESOLVED	Aarav Menon
2	10003	HIGH	Tonle Sap Fringe	Mekong Basin	17.41	OPEN	(null)
3	10002	MEDIUM	Tonle Sap Fringe	Mekong Basin	13.91	OPEN	(null)
4	10001	MEDIUM	Tonle Sap Fringe	Mekong Basin	10.66	OPEN	(null)

Figure 14. Q18 alert dashboard in SQL Developer: the 4 trustworthy alerts; alert 10000 resolved by the trigger.

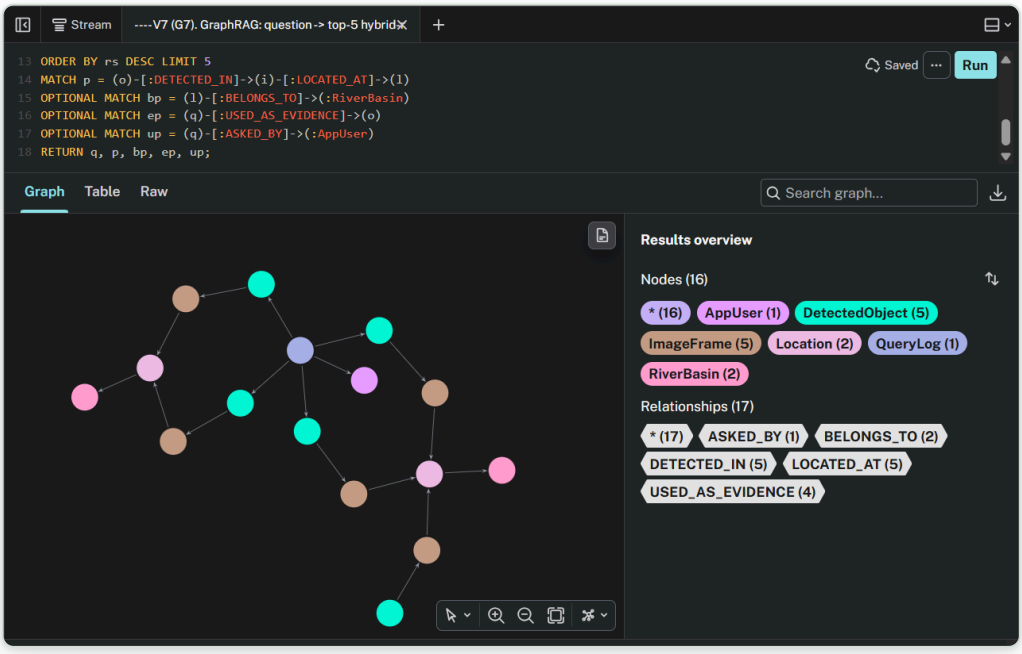


Figure 15. Neo4j GraphRAG path: question → top-5 evidence → image → location → basin.

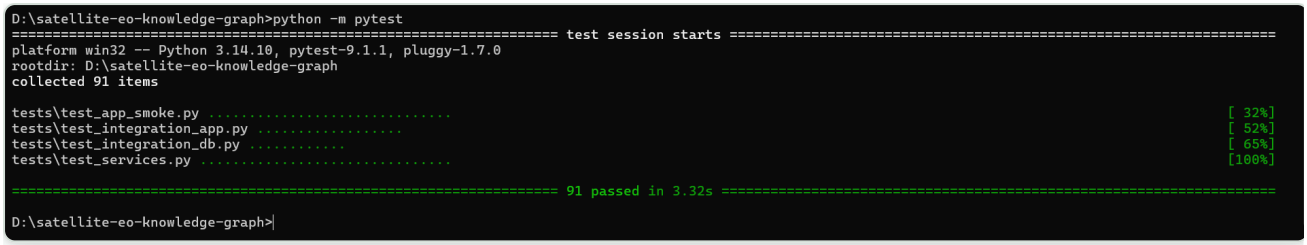


Figure 16. Automated test run: 91 passed in 3.32 s across the four test files.

8.4 Discussion

The relational design proved strongest for integrity: CHECK, UNIQUE and FK constraints plus triggers rejected every invalid operation in testing, and the PL/SQL package kept derived values consistent. Multi-hop questions needed up to six joins in SQL and a CONNECT BY query for lineage, whereas Neo4j expressed the flagship question as one path pattern and added native vector and spatial search, which Oracle XE lacks. The trust-aware design changed the outcome materially: 6 of 10 deforestation events were not alerted because their evidence was cloudy or uncertain, which prevents false alarms. The best overall design is therefore hybrid: Oracle as the system of record and Neo4j as the retrieval layer.

09 Conclusion, Limitations and Future Enhancements

The project delivered a BCNF-normalised 15-table Oracle database, a PL/SQL layer that builds temporal lineage, raises trust-aware alerts and computes answer confidence, a Flask application with nine database-connected features and two-level validation, and a Neo4j graph extension that reproduces the key results. The three ideas (temporal lineage, tunable hybrid retrieval and confidence propagation) are all implemented and visible in the application.

Limitations

- Synthetic data and 8-dimensional demonstration embeddings.
- Concept-based search (centroid of labelled examples) rather than free-text understanding by a language model.
- One river reference point per basin instead of basin polygons and river lines (Oracle XE has no spatial option).
- Lineage assumes one tree patch per image per location.
- No login or roles; maps and charts need internet (CDN); Oracle and Neo4j are loaded separately, not synchronised.

Future enhancements

- Real Sentinel-2 imagery with real detection models and learned image embeddings.
- Basin polygons and river lines in a spatial database for true distances.
- Object matching across dates using overlap and similarity, supporting several patches per image.
- A language model that writes the final answer from the retrieved evidence (full GraphRAG).
- Automatic Oracle → Neo4j synchronisation (change data capture), login and role-based access.

References

[1] D. Edge et al., "From local to global: A Graph RAG approach to query-focused summarization," arXiv:2404.16130, 2024.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474.

[3] M. C. Hansen et al., "High-resolution global maps of 21st-century forest cover change," *Science*, vol. 342, no. 6160, pp. 850–853, 2013.

[4] M. Drusch et al., "Sentinel-2: ESA's optical high-resolution mission for GMES operational services," *Remote Sens. Environ.*, vol. 120, pp. 25–36, 2012.

[5] C. J. Tucker, "Red and photographic infrared linear combinations for monitoring vegetation," *Remote Sens. Environ.*, vol. 8, no. 2, pp. 127–150, 1979.

[6] B.-C. Gao, "NDWI—A normalized difference water index for remote sensing of vegetation liquid water from space," *Remote Sens. Environ.*, vol. 58, no. 3, pp. 257–266, 1996.

[7] N. Gorelick et al., "Google Earth Engine: Planetary-scale geospatial analysis for everyone," *Remote Sens. Environ.*, vol. 202, pp. 18–27, 2017.

[8] P. P. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, 1976.

[9] E. F. Codd, "A relational model of data for large shared data banks," *Commun. ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[10] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.

[11] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[12] World Resources Institute, "Global Nature Watch." [Online]. Available: https://www.globalnaturewatch.org

[13] World Resources Institute, "Global Forest Watch becomes Global Nature Watch, expanding to cover more land ecosystems," press release. [Online]. Available: https://www.wri.org/news/release-global-forest-watch-becomes-global-nature-watch-expanding-cover-more-land-ecosystems

[14] Copernicus Data Space Ecosystem, "About the Browser." [Online]. Available: https://documentation.dataspace.copernicus.eu/Applications/Browser.html

[15] Pallets Projects, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com

[16] Oracle, "python-oracledb documentation." [Online]. Available: https://python-oracledb.readthedocs.io

[17] Neo4j, Inc., "Cypher Manual: Vector indexes." [Online]. Available: https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/

[18] Oracle, "Oracle Database 21c PL/SQL Language Reference." [Online]. Available: https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/

[19] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.